

Ironwood Guide

Constructing and validating a shielded payment

m@rek.onl

Abstract

This volume of *The Zcash Arboretum* constructs a private payment: the sender's preparation, the recipient's recovery, and the public checks that permit the payment to enter the accepted record. Orchard is the payment construction; Ironwood names a separately accounted use of that construction under the specified upgrade rules. It assumes the *Math Guide*'s field and curve constructions, the *Crypto Guide*'s commitments, signatures and encryption, the *Halo 2 Guide*'s proof contract, and the *Consensus Guide*'s branch-local ledger contract. Each derived object has an explicit type, purpose and consumer. The distinction between a specified upgrade and an activated network rule is maintained throughout. This exposition is non-normative.

Contents

1	The shielded payment contract	3
1.1	Protocol, pool and rule set	3
1.2	Types and a continuing example	3
2	Keys and viewing capabilities	4
2.1	Expansion and spend authority	4
2.2	Viewing keys and addresses	4
2.3	Internal scope and capability limits	5
3	Notes and their commitments	5
3.1	The committed note	6
4	Nullifiers and authenticated existence	6
4.1	The nullifier equation	6
4.2	Reconstructing a note from a seed	7
4.3	The note tree	7
4.4	Anchors and zero-value inputs	8
4.5	Action value and authorisation keys	8
5	Note delivery and recovery	8
5.1	Encrypting the note	9
5.2	Outgoing recovery	9
5.3	Recipient acceptance	9
6	Actions and the proof relation	10
6.1	Public encoding and private witness	10
6.2	How the circuit implements the relation	11
7	Balance, digests, and authorisation	12
7.1	Value commitments and integer conservation	12
7.2	Spend authorisation	12
7.3	The complete version-5 byte envelope	13
7.4	Version-5 effecting and authorising digests	14
7.5	The specified version-6 extension	14
8	Pool rules and complete validation	15
8.1	The conditional seal	15
8.2	The node's checklist	16
9	One payment, end to end	16
10	Security boundaries	17
10.1	What is mathematical	17
10.2	What is assumed or imported	17
10.3	What remains visible or unavailable	18

1 The shielded payment contract

The public ledger needs an authorised, value-preserving state transition. A ledger, note, pool, anchor and nullifier have the interfaces introduced in the *Consensus Guide*, “Ledger state and transaction acceptance”. For the concrete primitives we use the *Crypto Guide*, “Groups, commitments, and Sin-semilla”, “RedPallas signatures”, and “Key agreement and note encryption”. A proof’s public inputs, private witness and acceptance relation are constructed in the *Halo 2 Guide*, “From a relation to a circuit”. A private payment must supply evidence of that transition without naming the input note or publishing its value and recipient. Four mechanisms divide the work: commitments represent hidden notes; nullifiers identify repeated spends; proofs establish the private arithmetic relation; and signatures authorise the transaction. Encryption delivers the new notes. The node’s branch-local state supplies the final existence and uniqueness context.

This division matters. An accepting proof is not, by itself, a valid transaction. It need not establish possession of a spending secret, correct note delivery, absence of a previous nullifier, or the validity of another pool’s component. The full acceptance contract of the *Consensus Guide*, “Ledger state and transaction acceptance”, remains in force.

1.1 Protocol, pool and rule set

The *Orchard protocol* is the shared cryptographic construction. A *pool* is a separate accounting and state namespace: its own note tree, anchors, nullifier set and public value balance. The original Orchard pool and the specified Ironwood pool use the same key and address family. Sharing addresses does not merge their trees or spend histories.

An *Action* is a record pairing one old note with one new note. Either side can have value zero to fill an otherwise unused slot. A *bundle* collects one pool’s ordered Actions and their shared validation data. These are the units of construction throughout the book. The *lead byte* is the first byte of a delivered note’s plaintext and identifies its format. A transaction *digest* is a hash of prescribed transaction data. A *cross-address restriction* requires the old and new receivers within each Action to be equal.

Two rule profiles will be explicit:

- The Orchard-pool construction uses note plaintext lead byte 0x02 and the version-5 transaction digest structure.
- The specified NU6.3 profile adds Ironwood, lead byte 0x03, version-6 digests and the cross-address restriction used to seal the original Orchard pool.

The latter applies only where and when its defining consensus rules activate. The guide does not infer a network’s activation state from a crate version or a ZIP status.

The principal source pin is zips commit 69610984; the implementation references are orchard 0.15.0, librustzcash commit e30517e4, and the orchard development tree bef8a27 where explicitly named. The public ZIP 229, ZIP 258 and ZIP 2005 describe the specified Ironwood changes. The version-6 details below use the published ZIP 229 revision consulted on 6 September 2026, including its explicit version group and reserved-bit rules; those later clarifications are not attributed to the older local ZIP commit. Their status is not an activation certificate.

1.2 Types and a continuing example

The two primes and canonical encodings are constructed in the *Math Guide*, “Pasta curves and their encodings”. Let $p = p_{\text{Pallas}}$ and $r = p_{\text{Vesta}}$. Pallas points form a prime-order group of order r , with coordinates in $\mathbb{F}_p$; the Action circuit is over $\mathbb{F}_p$. In particular $p < r$. Converting a canonical integer in $[0, p)$ into a scalar in $\mathbb{F}_r$ is an injective conversion of integer representatives, not a field homomorphism. For example, $(p - 1) + 1 = 0$ in $\mathbb{F}_p$, whereas the same integer representatives sum to $p \neq 0$ in $\mathbb{F}_r$.

The notation $\text{LE}_k(x)$ denotes k little-endian bits. When a byte hash consumes such a string, pack each consecutive eight bits into a byte, least significant bit first. We write $\text{enc}_n(x)$ for an unsigned integer's n -byte little-endian encoding; for a field or scalar at width 32, encode its canonical representative. Signed encodings use the explicitly marked two's-complement convention. Write P^* for a compressed 32-byte Pallas point encoding. The coordinate extractor is $\text{Extract}(P) = x(P)$ for an affine point and $\text{Extract}(\mathcal{O}) = 0$. It is not a point encoding: P and $-P$ have the same extraction.

The continuing example spends an 80,000-zatoshi note, pays 50,000, creates 20,000 change and pays a 10,000 fee. It uses two Actions: one pairs the real input with the payment; one pairs a zero-valued dummy input with change. Their private net values are +30,000 and -20,000. These are checked arithmetic examples, not a claim that the text contains a generated production proof.

2 Keys and viewing capabilities

An address lets another party create a payable note. A viewing key discovers and interprets notes. A spending key authorises their consumption. The construction must bind these capabilities together without publishing the secret authorising scalar.

2.1 Expansion and spend authority

For a 32-byte secret s and byte string t , define

$$\text{PRFexpand}_s(t) = \text{BLAKE2b}_{512}(\text{Zcash_ExpandSeed}, s || t).$$

This uses personalised *unkeyed* BLAKE2b with a secret prefix, not BLAKE2b's native keyed mode. Its required pseudorandom-function property is an assumption discussed in the *Crypto Guide*, "Hash functions and key derivation". Define ToBase and ToScalar by reading the 64-byte output little-endian and reducing modulo p and r , respectively. The reduction bias of a uniform 512-bit input is bounded by the target modulus divided by 2^{512} ; PRF-generated inputs add the computational PRF distinction.

From spending key sk derive

$$\begin{aligned} a &= \text{ToScalar}(\text{PRFexpand}_{\text{sk}}(0x06)), \\ \text{nk} &= \text{ToBase}(\text{PRFexpand}_{\text{sk}}(0x07)), \\ r_{\text{ivk}} &= \text{ToScalar}(\text{PRFexpand}_{\text{sk}}(0x08)). \end{aligned}$$

Let $G = \text{GroupHash}^{\text{Pallas}}(\text{z.cash:Orchard}, G)$, using the group-hash construction in the *Crypto Guide*, "Hash functions and key derivation". Reject $a = 0$. If $[a]G$ has odd canonical y coordinate, set $\text{ask} = -a$; otherwise set $\text{ask} = a$. Then $\text{ak} = [\text{ask}]G$ has even y and is nonidentity.

The canonical sign permits $\text{Extract}(\text{ak})$ to identify ak in the next construction. It does not mean that every Pallas point can be encoded by its x coordinate without an additional convention.

2.2 Viewing keys and addresses

The full viewing key is $\text{fvk} = (\text{ak}, \text{nk}, r_{\text{ivk}})$. Instantiate Sinsemilla's short commitment under domain $\text{z.cash:Orchard-CommitIvk}$:

$$\begin{aligned} \text{ivk} &= \text{Commit}_{r_{\text{ivk}}}^{\text{ivk}}(\text{Extract}(\text{ak}), \text{nk}), \\ &= \text{ShortCommit}_{\text{z.cash:Orchard-CommitIvk}, r_{\text{ivk}}}(\text{LE}_{255}(\text{Extract}(\text{ak})) || \text{LE}_{255}(\text{nk})). \end{aligned}$$

The construction and its exceptional value $\perp$ are defined in the *Crypto Guide*, "Groups, commitments, and Sinsemilla". Reject an output of zero or $\perp$ when deriving a receiving key. A successful $\text{ivk} \in [1, p)$ converts to a nonzero Pallas scalar by this canonical-integer convention.

Let

$$R = \text{PRFexpand}_{\text{enc}_{32}(r_{\text{ivk}})}(0x82\|ak^*\| \text{enc}_{32}(nk)),$$

$$dk = R[0 : 32], \quad ovk = R[32 : 64].$$

The byte slicing uses a half-open interval. A single expansion output supplies both keys. The incoming viewing key in the complete wallet sense is (dk, ivk) : ivk performs decryption, while dk generates the wallet's indexed diversifiers.

A diversifier d is an 11-byte string. Define

$$g_d = \text{DiversifyHash}(d), \quad pk_d = [ivk]g_d, \quad \text{addr} = (d, pk_d).$$

The diversification hash is $\text{GroupHash}^{\text{Pallas}}(z.\text{cash}:\text{Orchard-gd}, d)$, with its identity output replaced by the fixed result for the empty message in the same domain. Thus the protocol's diversified base is nonidentity. The raw receiver is $d\|pk_d^*$, 43 bytes. Canonical point decoding and nonidentity checks are necessary when accepting an address from another party.

2.3 Internal scope and capability limits

A wallet can separate externally issued addresses from its internal change addresses while retaining the same spend-authorising scalar. Its internal full viewing key keeps ak and nk and replaces the commitment randomness by

$$r_{\text{ivkint}} = \text{ToScalar}(\text{PRFexpand}_{\text{enc}_{32}(r_{\text{ivk}})}(0x83\|ak^*\| \text{enc}_{32}(nk))).$$

Apply the preceding incoming and outgoing derivations to that tuple. Standard spending-key validity includes nonzero, nonexceptional incoming keys for both external and internal scopes. This construction is ZIP 32's Orchard internal key derivation, implemented in `orchard/src/keys.rs`.

Capability	What possession permits
Receiver	Create payments; it is not an authorisation secret.
Incoming key	Discover and decrypt matching incoming notes; derive addresses when the diversifier key is included.
Full viewing key	Additionally compute nullifiers of known notes and derive the specified outgoing recovery keys; it does not sign spends.
Spending authority	Produce spend signatures; with note and state data, construct complete spends.

An outgoing viewing key recovers only outputs whose sender used that key for outgoing recovery. A sender may deliberately omit that recovery capability. A full viewing key therefore does not reveal every possible outgoing payment of a malicious or independently configured spender. Nor does possession of a public receiver prove that its advertised owner knows the spending secret: the sender can validate encodings, but cannot perform a recipient's private key-generation checks.

3 Notes and their commitments

A note is the private description of value that a later proof will consume. Its commitment must bind the amount and receiving capability while concealing them from a chain observer.

3.1 The committed note

Write a note as

$$\mathbf{n} = (d, \text{pk}_d, v, \rho, \psi, \text{rcm}), \quad 0 \leq v < 2^{64}, \quad \rho, \psi \in \mathbb{F}_p, \quad \text{rcm} \in \mathbb{F}_r.$$

The diversified base g_d is reconstructed from d . The amount's circuit range is 64 bits; transaction and chain money bounds are separate checks, not a reason to omit this range constraint.

Define its 1086-bit message

$$m(\mathbf{n}) = g_d^* \parallel \text{pk}_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi),$$

where compressed point bytes are interpreted as the corresponding 256 bits. The note commitment and its published extraction are

$$\text{cm} = \text{NoteCommit}_{\text{rcm}}(m(\mathbf{n})), \quad \text{cmx} = \text{Extract}(\text{cm}).$$

The commitment is `SinsemillaCommit` under `z.cash:Orchard-NoteCommit`. The lower primitive pads the final partial ten-bit chunk according to its fixed-length convention; an application must not insert a byte-alignment pad between ρ and ψ . The point commitment is used in the nullifier equation, while only cmx is appended to the note tree.

For genuinely uniform independent rcm , the randomisation component has the hiding property proved for the commitment primitive. In honest notes rcm is derived from a seed and other note data. Their hiding argument is computational and depends on the expansion function's security; it is not a claim of unconditional hiding of the whole transaction.

4 Nullifiers and authenticated existence

A commitment does not disappear when spent. Removing the input leaf would reveal the spend-to-note link. The ledger instead keeps all commitments and records a separate public tag on consumption.

4.1 The nullifier equation

Let

$$K = \text{GroupHash}^{\text{Pallas}}(\text{z.cash:Orchard}, K), \quad F_{\text{nk}}(\rho) = \text{Poseidon}(\text{nk}, \rho).$$

The Poseidon instantiation is the fixed-length-two P128Pow5T3 construction over $\mathbb{F}_p$, with three state words and two input field elements, constructed in the *Crypto Guide*, “Hash functions and key derivation”. For note $\mathbf{n}$ with point commitment cm ,

$$\text{nf} = \text{Extract}([a]K + \text{cm}), \quad a = \text{repr}_{[0,p)}(F_{\text{nk}}(\rho) + \psi). \quad (1)$$

The sum is first reduced in $\mathbb{F}_p$; its canonical integer a then converts to $\mathbb{F}_r$ for scalar multiplication. Adding instead in $\mathbb{F}_r$ would be a different function.

For fixed note and key components this equation is deterministic. With commitment and key-binding security, a valid spend cannot choose a new tag for the same nonzero note. The public node rejects any repeated tag in the relevant pool, including repeats in the same transaction or block. Determinism alone does not prove that arbitrary maliciously chosen notes have collision-resistant nullifiers; the complete argument also uses the commitment construction and how note uniqueness is established.

Every new note uses $\rho_{\text{new}} = \text{nf}_{\text{old}}$ from its own Action. Since accepted nullifiers are unique within a pool, accepted output notes have distinct creation identifiers there. This includes zero-value outputs. Across two pools there are separate nullifier namespaces; no global uniqueness claim follows merely from this rule.

4.2 Reconstructing a note from a seed

The sender draws a fresh 32-byte r_{seed} . Its first required input is ρ : the nullifier of the old side of the same Action, just constructed in §4.1. A dummy old side still produces a nullifier, so this rule applies even when no real note is consumed.

Derive

$$\begin{aligned} \text{esk} &= \text{ToScalar}(\text{PRFexpand}_{r_{\text{seed}}}(\text{0x04} \parallel \text{enc}_{32}(\rho))), \\ \psi &= \text{ToBase}(\text{PRFexpand}_{r_{\text{seed}}}(\text{0x09} \parallel \text{enc}_{32}(\rho))). \end{aligned}$$

Redraw the seed if the ephemeral scalar esk is zero. For an Orchard lead-byte-0x02 note,

$$\text{rcm} = \text{ToScalar}(\text{PRFexpand}_{r_{\text{seed}}}(\text{0x05} \parallel \text{enc}_{32}(\rho))).$$

For a specified Ironwood lead-byte-0x03 note, instead compute

$$\begin{aligned} t_{\text{rcm}} &= \text{0x0b} \parallel g_d^* \parallel pk_d^* \parallel \text{enc}_8(v) \\ &\quad \parallel \text{enc}_{32}(\rho) \parallel \text{enc}_{32}(\psi), \\ \text{rcm} &= \text{ToScalar}(\text{PRFexpand}_{r_{\text{seed}}}(t_{\text{rcm}})). \end{aligned}$$

The preimage after the seed has 137 bytes: $1 + 32 + 32 + 8 + 32 + 32$. Its fields are byte encodings, unlike the 255-bit field encodings inside $m(\mathbf{n})$. The dependency order is forced: derive ψ before the version-3 rcm . Both versions use the same esk and ψ formulas.

The version-3 change binds the seed derivation to the other note fields as specified in ZIP 2005. It does not make the payment proof, signatures or key agreement independent of discrete logarithms. No alternate recovery protocol is needed to understand these bytes.

If note commitment evaluation fails with $\perp$, honest construction retries with fresh randomness. The formal Action relation’s treatment of exceptional Sinsemilla cases is different and is stated explicitly in §6.1.

4.3 The note tree

The note tree has depth 32 and leaf capacity 2^{32} . Leaves are published cmx values in transaction and Action order. Unfilled leaves contain the fixed sentinel $2 \in \mathbb{F}_p$.

It is convenient to count hash height from the leaves. For $j = 0, \dots, 31$, first compute

$$h_j(l, r) = \text{ShortHash}_{\text{z.cash:Orchard-MerkleCRH}}(\text{LE}_{10}(j) \parallel \text{LE}_{255}(l) \parallel \text{LE}_{255}(r)).$$

Then set $H_j(l, r) = 0$ if $h_j(l, r) = \perp$, and $H_j(l, r) = h_j(l, r)$ otherwise. This conversion is the concrete MerkleCRH rule, also implemented in `orchard/src/tree.rs`; an exceptional hash is not a separate non-field object stored in the public tree. Each preimage has 520 bits, exactly 52 ten-bit chunks. The first combine above leaves uses $j = 0$; the root combine uses $j = 31$. The Sinsemilla short-hash construction, including its exceptional semantics, is in the *Crypto Guide*, “Groups, commitments, and Sinsemilla”. The required exceptional-case hardness prevents an efficient adversary from deliberately obtaining such a zero hash.

For a leaf position $i \in [0, 2^{32})$, let i_j be bit j of i and s_j the sibling at height j . Starting with $r_0 = \text{cmx}$, compute

$$r_{j+1} = \begin{cases} H_j(r_j, s_j), & i_j = 0, \\ H_j(s_j, r_j), & i_j = 1. \end{cases} \quad (2)$$

The path has 32 siblings, or 1024 bytes of field encodings, and a position determining left versus right. The final value r_{32} is the root.

The proof keeps the position and siblings private. Its public anchor identifies a candidate tree, not the spent leaf. This does not make every leaf an equally plausible input: publicly identifiable dummy outputs, already explained spends and external metadata can reduce anonymity.

4.4 Anchors and zero-value inputs

All Actions in one bundle use its common public anchor. The node checks that it is eligible in that pool on the candidate branch. Under the modern rule this is an earlier block’s final treestate, not a tree invented by the prover and not an intermediate state of the same block.

In the ordinary nonexceptional path calculation, the circuit gates membership by old value:

$$v_{\text{old}}(r_{32} - \text{anchor}) = 0.$$

For a nonzero 64-bit old amount, the factor is nonzero in $\mathbb{F}_p$, so the root must equal the anchor. A zero-valued old side may have no membership witness to an accepted note. This permits dummy spends and shielding. It does not permit a positive-value invented input. The Action still has other constraints, a nullifier and a spend signature; dummy does not mean an arbitrary unvalidated byte record.

The formal circuit contract has two additional allowances, stated in the protocol specification’s “Merkle Path Validity” and “Action Statement” notes. Intermediate hash inputs can use 255-bit representatives that reduce to the assigned field values without being their canonical representatives. The membership check may also pass if a hash on the path, including the root, is zero. Write PathOK for this permitted predicate, rather than silently identifying it with an ordinary canonical path recomputation. The zero-hash allowance models incomplete Sinsemilla addition. Its security argument needs the exceptional-case hardness assumption; honest public tree computation still uses the canonical function H_j above.

4.5 Action value and authorisation keys

Before encrypting an output, construct two public Action fields that the delivery and proof algorithms will consume. The first hides its net value. Using the commitment construction in the *Crypto Guide*, “Groups, commitments, and Sinsemilla”, define

$$V = \text{GroupHash}^{\text{Pallas}}(z.\text{cash}:\text{Orchard-cv}, v), \quad R = \text{GroupHash}^{\text{Pallas}}(z.\text{cash}:\text{Orchard-cv}, r).$$

Their required independence means no efficiently known discrete-log relation, not linear independence in a cyclic group. For Action i , let $d_i = v_{\text{old},i} - v_{\text{new},i}$, sample $\text{rcv}_i \in \mathbb{F}_r$, and publish the net value commitment

$$\text{cv}_i^{\text{net}} = [d_i]V + [\text{rcv}_i]R.$$

A negative integer coefficient denotes the negative of the corresponding positive multiple. The values remain integers even though multiplication uses their residues in $\mathbb{F}_r$.

The second field is a randomised spend-authorisation key. Sample $\alpha \in \mathbb{F}_r$ and set

$$\text{rsk} = \text{ask} + \alpha, \quad \text{rk} = \text{ak} + [\alpha]G = [\text{rsk}]G.$$

Resample if $\text{rsk} = 0$, equivalently if $\text{rk} = \mathcal{O}$, since consensus prohibits that public key. The circuit will connect rk to the old note’s receiving authority; a later signature will demonstrate approval under rsk . The zero-valued dummy old side uses a fresh valid spending key of the constructor’s choice and follows the same construction.

5 Note delivery and recovery

The proof can establish that an output commitment contains a well-formed note without giving its recipient the information needed to spend it. Delivery is a separate protocol. A malicious sender can create an undeliverable output; ordinary transaction validation does not promise otherwise.

5.1 Encrypting the note

For lead byte ℓ , the note plaintext is the fixed 564-byte string

$$\text{np} = \ell || d || \text{enc}_8(v) || \text{rseed} || \text{memo},$$

where the memo occupies exactly 512 bytes. The first 52 bytes contain all note reconstruction data except ρ , which is taken from the Action nullifier. The format supplies the recipient's diversified base through d ; the recipient recovers pk_d from its viewing key.

Using the nonzero ephemeral scalar already derived, compute

$$\begin{aligned} \text{epk} &= [\text{esk}]g_d, & S &= [\text{esk}]pk_d, \\ k_{\text{enc}} &= \text{BLAKE2b}_{256}(\text{Zcash_OrchardKDF}, S^* || \text{epk}^*). \end{aligned}$$

Encrypt np with ChaCha20-Poly1305, nonce 0^{96} and empty associated data. The result C_{enc} has 580 bytes, including the 16-byte authentication tag. The *Crypto Guide*, “Key agreement and note encryption”, constructs the cipher and its security contract.

The fixed nonce is safe only under the construction's key-freshness conditions. Reusing seed and context can reuse a key; the nonce is not a substitute for fresh output randomness. Honest key generation and seed sampling are therefore part of the encryption security assumptions.

The recipient computes the same shared point because

$$[\text{ivk}]epk = [\text{ivk}][\text{esk}]g_d = [\text{esk}]pk_d.$$

This is a group identity. Confidentiality of the resulting ciphertext is a computational claim under key-agreement, key-derivation and cipher assumptions, not a consequence of the identity alone.

5.2 Outgoing recovery

For Action net value commitment cv^{net} , derive

$$k_{\text{out}} = \text{BLAKE2b}_{256}(\text{Zcash_Orchardock}, \text{ovk} || ((\text{cv}^{\text{net}})^*) || \text{enc}_{32}(\text{cmx}) || \text{epk}^*).$$

The commitment was constructed in §4.5. Encrypt $pk_d^* || \text{enc}_{32}(\text{esk})$ using the same authenticated-encryption-with-associated-data (AEAD) convention. The 64-byte plaintext yields an 80-byte C_{out} . An auditor with ovk can recover pk_d and esk , derive the incoming key material and recover the note.

The sender can instead choose a fresh random outgoing key and random outgoing plaintext, withholding outgoing recovery. The ciphertext is not proof that a particular ovk was used. Nor does a decryptable memo authenticate a real-world sender identity.

5.3 Recipient acceptance

For each candidate Action and each incoming key, perform the following:

1. Decode the ephemeral point canonically and apply the required nonidentity check. Compute $[\text{ivk}]epk$ and the incoming KDF.
2. Open the full incoming ciphertext with AEAD verification. Reject failure and a lead byte not allowed for this pool and format.
3. Parse d, v and rseed , and take $\rho = \text{nf}$ from the same Action. Compute g_d and $pk_d = [\text{ivk}]g_d$.
4. Recompute esk , ψ and the correct version's rcm . Check $[\text{esk}]g_d = \text{epk}$.
5. Recompute the note commitment and require its extraction to equal the published cmx . Only then record a detected note.

The ephemeral-key check binds the recovered seed to the transmitted ephemeral key. The commitment check binds the recovered note fields to the published output. A successful AEAD opening alone is insufficient.

Compact scanning omits the memo and authentication tag and therefore uses the stream-cipher prefix plus these reconstruction checks, not a successful full AEAD verification. It discovers candidate notes against supplied chain data; it does not itself validate that chain.

Shielded coinbase has special public recovery rules. Its prescribed outgoing viewing key is zero, allowing validators and any observer to recover the output plaintext and check its required format and values. The ordinary incoming privacy discussion must not be applied to coinbase outputs without this exception.

6 Actions and the proof relation

An *Action* pairs one old side with one new side. The pairing fixes the new note's creation identifier. It does not disclose to a chain observer which old note is used, or whether either side has zero value.

6.1 Public encoding and private witness

The Action wire record is

$$(cv^{net}, nf, rk, cmx, epk, C_{enc}, C_{out}).$$

The first five fields are each 32 bytes; the ciphertexts are 580 and 80 bytes. Thus one Action record is 820 bytes. Its spend-authorisation signature is a separate 64-byte field. The bundle adds shared flags, signed value balance, anchor, aggregate proof and a *binding signature*, the signature enforcing its combined value balance.

For the specified cross-address circuit, the public instance vector is

$$(\text{anchor}, x(cv^{net}), y(cv^{net}), nf, x(rk), y(rk), cmx, e_s, e_o, \delta).$$

The point-coordinate representation assigns the identity the pair (0,0) where it is permitted; other points use their affine coordinates. External parsing must match it. Consensus separately prohibits identity rk , while the value commitment can be identity. There are ten field elements. The flags e_s, e_o enable nonzero spends and outputs. The flag δ disables transfers between different receivers. Consensus and the selected verification key determine which flag values apply; they are not free choices of the witness.

The witness contains the old side's $g_d, pk_d, v, \rho, \psi, rcm$ and point commitment, the new side's g_d, pk_d, v, ψ, rcm , the old note's membership path and position, ak, nk, r_{ivk} , a spend-key randomiser α , and value-commitment randomness rcv . It does *not* contain d , $rseed$, esk , the ciphertexts or ask . This is why proving can be separated from authorising a spend. The proof does not check the honest seed-expansion or diversification-hash procedures; recipient reconstruction checks the delivered note against them. The original circuit's nine-element instance omits δ and the additional cross-address condition.

Definition 6.1 (Action relation). The typed witness and public inputs satisfy:

- A1.** The old point commitment equals the note commitment computed from its witnessed fields, or that computation returns $\perp$.
- A2.** With $\rho_{new} = nf$, the new commitment's extraction equals public cmx , or that computation returns $\perp$.
- A3.** Either $v_{old} = 0$ or the old membership witness satisfies PathOK for the public anchor, including its explicitly permitted representation and exceptional-case semantics.

A4. Both amounts are in $[0, 2^{64})$ and $cv^{net} = [v_{old} - v_{new}]V + [rcv]R$.

A5. The old fields and nullifier key give public nf by (1).

A6. The randomised spend key satisfies $rk = ak + [\alpha]G$.

A7. The old receiving key satisfies $pk_{d_{old}} = [ivk]g_{d_{old}}$ with $ivk = \text{Commit}_{r_{ivk}}^{ivk}(\text{Extract}(ak), nk)$, or that short-commitment evaluation returns $\perp$.

A8. The spend gate satisfies $v_{old}(1 - e_s) = 0$.

A9. The output gate satisfies $v_{new}(1 - e_o) = 0$.

A10. If $\delta = 1$, both $g_{d_{old}} = g_{d_{new}}$ and $pk_{d_{old}} = pk_{d_{new}}$ hold as full points.

Witness typing requires ak , both diversified bases and both diversified transmission keys to be nonidentity Pallas points. Unlike honest key generation, the Action witness permits either sign of ak . Its incoming-key commitment uses only $\text{Extract}(ak)$; knowledge of the authorising secret up to sign is sufficient. Scalar representatives used only in an isolated multiplication may be non-canonical when reduction gives an equivalent witness. The nullifier scalar and incoming scalar, whose values also participate elsewhere, need the prescribed canonical decompositions. Field, value-range and boolean constraints are part of the relation.

The Sinsemilla $\perp$ alternatives are real weakenings of the simple mathematical equalities, not successful honest note construction. Excluding adversarial exploitation requires the primitive's exceptional case hardness argument under its generator assumptions. A proof of the weakened relation alone does not justify dropping these alternatives from a security reduction.

The last condition consists of four field equations:

$$\delta(x(P_{old}) - x(P_{new})) = 0, \quad \delta(y(P_{old}) - y(P_{new})) = 0$$

for $P = g_d, pk_d$. Equality of extracted coordinates alone would permit opposite points and would not express equality of receivers.

6.2 How the circuit implements the relation

The *Halo 2 Guide*, “From a relation to a circuit”, constructs the constraint language and its connection to a proof. A *gadget* is a reusable group of circuit constraints implementing one operation. A *limb* is a bounded positional digit, such as a ten-bit digit in base 2^{10} . Here the private integer v is represented by bounded limbs. Range lookups and reconstruction equations ensure that the field value is the intended 64-bit integer. A signed difference can be represented by a bounded magnitude and sign in $\{-1, +1\}$, so fixed-base multiplication commits to the integer difference with its proper sign.

The note-commitment gadget decomposes its assigned points and field values into the constrained bit sequence corresponding to §3.1, subject to the specification's representation allowances and including the 255-bit boundaries. In *fixed-base multiplication* the point is fixed by the circuit; in *variable-base multiplication* it is supplied as part of the instance or witness. Elliptic-curve constraints implement the fixed-base multiplications for commitments and randomised keys, and variable-base multiplication for the old transmission key. The Poseidon gadget implements $F_{nk}(\rho)$; the tree gadget implements the 32 hashes of (2). Copy constraints tie repeated values to the same assigned witness, rather than allowing independently chosen copies.

A field equation only proves its advertised integer statement after the relevant range and canonicity conditions have been established. Similarly, a scalar-multiplication gadget must constrain its base to the intended point. These are obligations of the actual circuit, not facts supplied by the notation $[a]P$.

The source implementation is orchard 0.15.0, `src/circuit.rs`, with note and incoming-key gadgets in `src/circuit/note_commit.rs` and `src/circuit/commit_ivk.rs`. The instance offsets

put δ at index 9. Circuit versions choose different verification keys. A verifier must select by the applicable consensus rules, not let the prover select a weaker historical key.

The aggregate proof covers all Actions of a bundle with their ordered instances. It does not verify the blockchain's preceding proofs. The fixed construction uses 2^{11} circuit rows per Action; aggregating m Actions gives the specified proof byte length $2720 + 2272m$. For two Actions this is 7264 bytes. These implementation parameters describe the pinned proof format, not a universal lower bound on zero-knowledge proof sizes.

7 Balance, digests, and authorisation

The Action relation relates each hidden old value to its paired new value. The transaction needs a single public net balance and explicit approval of the transaction data. Two signature roles complete those obligations.

7.1 Value commitments and integer conservation

Using the already constructed cv_i^{net} , d_i , V and R , for the bundle's signed public balance b compute

$$\text{bvk} = \sum_i cv_i^{\text{net}} - [b]V.$$

If $b = \sum_i d_i$ as integers, the value terms cancel and

$$\text{bvk} = [\text{bsk}]R, \quad \text{bsk} = \sum_i rcv_i \pmod{r}.$$

The binding signature uses the RedPallas binding base R and the transaction signature digest. Honest correctness is the displayed group identity plus signature correctness.

Conversely, an unbalanced set of extracted commitment openings leaves

$$\text{bvk} = [D]V + [\sum_i rcv_i]R, \quad D = \sum_i d_i - b.$$

A sound balance argument must connect a valid binding signature to knowledge or unforgeability under this related public key, and then to the inability to know a nontrivial relation between V and R . It is not valid to infer knowledge of bsk merely from commitment binding. The required signature and proof composition assumptions are recorded in §10.

Finally, a group equation yields equality modulo r . The transaction's bounds lift it to integer equality. With fewer than 2^{16} Actions, 64-bit input and output ranges, and $|b| \leq 2.1 \cdot 10^{15}$,

$$|D| \leq m(2^{64} - 1) + |b| < 2^{81} < r.$$

Thus $D = 0 \pmod{r}$ implies $D = 0$ as an integer. The Action count bound is also far looser than what the block-size limit permits. Omitting this step would leave a modular conservation claim rather than a monetary one.

7.2 Spend authorisation

The randomised key (rsk, rk) was constructed in §4.5. The circuit proves its public-key equation. The spender then signs the transaction digest under rsk using RedPallas SpendAuth. The exact nonce generation, challenge hash, encoding checks and verification equation are constructed in the *Crypto Guide*, "RedPallas signatures". With nonce point R_s and challenge c , the verification identity has the form $[s]G = R_s + [c]\text{rk}$.

Fresh randomisation makes rk appear unrelated to ak under the stated key distribution, but does not hide every other transaction field. The authorising signature says that the party controlling the

corresponding secret approved the signed digest. The proof connects that key to the old note’s receiving capability. Neither fact alone is the complete spending-authority argument.

A dummy spend also has a randomised key and a signature. Its key can be freshly generated by the transaction constructor; the membership gate permits its zero old value. Its nullifier must nevertheless be unused.

7.3 The complete version-5 byte envelope

Serialization and hashing are different algorithms: their field order need not be the same. Here is the complete serialization for the Orchard-only payment, using the CompactSize construction from the *Consensus Guide*, “Blocks and branch-local validation”.

First emit the five four-byte little-endian header fields:

$$0x80000005, \quad 0x26A7270A, \quad \beta, \quad 0, \quad \eta.$$

The high version bit is set as required; β is the target height’s consensus branch identifier. Lock time is zero in this example; η is the chosen valid expiry height, or zero where disabling expiry is permitted. These fields occupy 20 bytes.

Next emit four single zero bytes: transparent input count, transparent output count, Sapling spend count and Sapling output count. Their arrays are empty, and no conditional Sapling balance, anchor or binding signature is present. Version 5 has no Sprout component. For the nonempty Orchard bundle, emit in order:

1. CompactSize Action count m , followed by the m Action records of §6.1, each 820 bytes.
2. One flags byte, $e_s + 2e_o$, with its remaining bits zero under the version-5 encoding rules.
3. Signed eight-byte value balance b , then the 32-byte anchor.
4. CompactSize proof length $2720 + 2272m$, then exactly that many proof bytes.
5. The m spend signatures, each 64 bytes, in Action order.
6. The 64-byte binding signature.

When $m = 0$, only its zero count is present; all those additional Orchard fields are absent. In a general transaction the transparent and Sapling counts instead govern their complete compatibility components. Their specialized parser must consume and validate those components before returning control at the Orchard count.

For $m = 2$, the proof length 7264 has a three-byte CompactSize encoding. The full ordinary Orchard-only transaction therefore has

$$20 + 4 + 1 + 1640 + 1 + 8 + 32 + 3 + 7264 + 128 + 64 = 9165$$

bytes. This is the actual format’s size calculation, not a fabricated proof or transaction test vector.

7.4 Version-5 effecting and authorising digests

Write $H_t(m) = \text{BLAKE2b}_{256}(t, m)$ for a fixed 16-byte personalisation t . For all Orchard Actions in order, form

$$\begin{aligned} D_c &= H_{\text{ZTxIdOrcActCHash}} \left(\left\| \left(\text{enc}_{32}(\text{nf}_i) \parallel \text{enc}_{32}(\text{cmx}_i) \parallel \text{epk}_i^* \parallel C_{\text{enc},i}[0 : 52] \right) \right\|_i \right), \\ D_m &= H_{\text{ZTxIdOrcActMHash}} \left(\left\| C_{\text{enc},i}[52 : 564] \right\|_i \right), \\ D_n &= H_{\text{ZTxIdOrcActNHash}} \left(\left\| \left((\text{cv}_i^{\text{net}})^* \parallel \text{rk}_i^* \parallel C_{\text{enc},i}[564 : 580] \parallel C_{\text{out},i} \right) \right\|_i \right). \end{aligned}$$

The component digest for a present Orchard bundle is

$$D_O = H_{\text{ZTxIdOrcHash}}(D_c \parallel D_m \parallel D_n \parallel \text{flags} \parallel \text{enc}_8^{\text{signed}}(b) \parallel \text{enc}_{32}(\text{anchor})).$$

An absent component instead hashes the empty string under its component personalisation. The hashes of an empty component and a nonempty zero-value component are not interchangeable.

The transaction identifier is

$$\text{txid} = H_{\text{ZcashTxHash}_\parallel \text{enc}_4(\beta)}(D_h \parallel D_t \parallel D_S \parallel D_O).$$

Here β is the consensus branch identifier. The header digest D_h hashes, under ZTxIdHeadersHash , the five four-byte fields in order: version header, version-group identifier, branch identifier, lock time and expiry height. The transparent and Sapling component digests are D_t and D_S . For the all-shielded example with neither component present they are the empty hashes under ZTxIdTranspaHash and ZTxIdSaplingHash . When those components are present, their complete ZIP 244 rules must be used; this is a compatibility interface, not a permission to commit only their totals.

For a transaction with no transparent inputs, the shielded signature digest equals this transaction identifier. With transparent inputs it uses ZIP 244's transparent signature digest, committing additionally to their spent output amounts and scripts and using the required `SIGHASH_ALL` mode for shielded authorisations. In particular, the signer needs authenticated previous-output data; an untrusted coordinator's claimed input total is not enough.

The Orchard authorising-data component is

$$A_O = H_{\text{ZTxAuthOrcHash}}(\text{proofs} \parallel \left\| \text{spendSig}_i \parallel \text{bindingSig} \right\|_i).$$

The root authorising digest uses personalisation $\text{ZTxAuthHash}_\parallel \text{enc}_4(\beta)$ and the transparent, Sapling and Orchard authorising digests in order. This digest is committed by the block's authorising-data tree, constructed in the *Consensus Guide*, "Authenticated block history".

Excluding signatures and proofs from txid avoids a signing cycle: signatures sign effecting data and subsequently enter a separate digest. Ciphertexts remain in effecting data. The proof need not check their content for the authorising signatures to bind their exact bytes.

7.5 The specified version-6 extension

Version 6 adds an Ironwood component after Orchard. Its root effecting and authorising digests have a fifth and fourth component child, respectively. For the affected shielded components the anchor moves from effecting to authorising data. Thus a changed anchor can require a new proof and authorising-data commitment without changing txid or, for the no-transparent-input case, the signed digest.

Its wire header changes to 0x80000006 and version group 0xD884B698; the latter is also the constant `V6_VERSION_GROUP_ID` at the pinned library source. After the complete Orchard component, serialize the Ironwood component using the same count, Action, flags, balance, anchor, proof-length, proof and signature ordering as above. Its count is always present; the additional fields are present only when that count is nonzero. Moving anchors between digest categories does *not* move their positions in this wire encoding. The flag interpretation is pool-specific and is fixed in §8.1.

The concrete changed personalisations in ZIP 229 are:

Digest	Version-6 personalisation
Orchard component	<code>ZTxIdOrchardH_v6</code>
Orchard authorising component	<code>ZTxAuthOrchaH_v6</code>
Ironwood component	<code>ZTxIdIronwd_H_v6</code>
Ironwood compact Actions	<code>ZTxIdIrnActCH_v6</code>
Ironwood memos	<code>ZTxIdIrnActMH_v6</code>
Ironwood noncompact Actions	<code>ZTxIdIrnActNH_v6</code>
Ironwood authorising component	<code>ZTxAuthIrnwdH_v6</code>

The Orchard Action-part personalisations remain their version-5 values. For each present version-6 bundle, append its anchor after the existing authorising-component fields and omit it from the effecting-component preimage. An absent bundle hashes empty input under its own domains. Sapling’s changed noncompact-spend and authorising domains are `ZTxIdSSpendNH_v6` and `ZTxAuthSapliH_v6`. Other version-5 components retain their defined structure.

Anchor movement does not detach a proof from its anchor: the anchor remains a proof instance, and the full authorising data remains committed by the block. It changes which commitment identifies that proof-bearing transaction representation.

8 Pool rules and complete validation

A bundle is one pool’s ordered Actions, shared flags, anchor, value balance, aggregate proof and signatures. The same transaction can contain both pool components. Each has its own anchor and binding key, while all authorisations bind the complete transaction digest.

8.1 The conditional seal

Under the specified NU6.3 rules, an Orchard-pool coinbase component must be empty, each Orchard Action disables cross-address transfer, and each transaction has $b_{\text{Orchard}} \geq 0$. These rules apply to the pool regardless of whether the transaction uses version 5 or version 6. They permit value to leave the pool and permit retained value to return to the same receiver; they prohibit net entry and ordinary transfers between different receivers within that pool.

For Ironwood, bit 2 is the specified `enableCrossAddress` flag, and the circuit input is $\delta = 1 - \text{enableCrossAddress}$. Bits 0 and 1 are spend and output enable flags. Reserved bits must be zero. The Orchard component’s bit 2 remains reserved as zero in the specified transaction rules; the node supplies the required cross-address restriction for that pool and branch. A parser accepting a bit pattern does not establish that the whole consensus verifier accepts it.

For a 70,000-zatoshi migration and a 10,000 fee,

$$b_{\text{Orchard}} = 80,000, \quad b_{\text{Ironwood}} = -70,000.$$

The public fee remains 10,000. Pool balances change in opposite directions, but neither pool’s tree acquires the other pool’s leaves. Keys and receiver encodings need not change merely because the pool namespace changes.

8.2 The node's checklist

For an ordinary transaction, a validating node:

1. Selects the network rules at the candidate block height and checks transaction version, version-group and branch identifiers, canonical lengths and all resource bounds.
2. Parses all field elements, points, signatures and proof encodings with their specified canonicity and identity conditions. Checks flags and the required nonempty input/output structure.
3. Builds each pool's ordered public instances, including its anchor and the branch-required cross-address semantics. Verifies the aggregate proof under the selected verification key.
4. Recomputes the correct version's transaction signature digest. Verifies every spend signature and each bundle's binding signature under its recomputed binding key.
5. Checks each anchor against eligible earlier final treestates in the correct pool and branch. Rejects repeated nullifiers within the transaction, earlier transactions in the block, or the parent state.
6. Validates every other component under its own rules. Checks transparent input availability and maturity, lock time, expiry, money ranges, fee, pool restrictions and resulting chain balances.
7. Proposes pool-local nullifier insertions and commitment appends in their defined order. Checks capacity and commits them only with the successfully validated whole block.

Coinbase replaces the ordinary spend and fee conditions with its special issuance and recovery rules. A node still verifies its applicable proofs, bindings and public accounting.

The ledger uses explicit lookup for nullifier freshness. In principle a different protocol could prove nonmembership relative to a committed state. It is inaccurate to say that global-history checks can never be represented in a proof; that is simply not how this acceptance contract is divided.

9 One payment, end to end

Alice has a detected, unspent 80,000-zatoshi note and its path to an eligible anchor. Bob supplies a validated receiver. Alice also derives an internal receiver for change. Choose a pool and branch profile that permit both resulting recipients; the sealed original Orchard pool would not permit this ordinary cross-address transfer after its seal.

1. Reconstruct the old note commitment and nullifier from its recorded seed, creation identifier and keys. Confirm locally that its path gives the intended anchor.
2. Construct a fresh zero-valued dummy old note, with its own valid key material and nullifier. The two nullifiers must differ and neither may already be spent in the pool.
3. Pair the real old side with Bob's 50,000 output and the dummy side with Alice's 20,000 output. Each output takes its own paired old nullifier as ρ .
4. Sample fresh output seeds and derive note randomness in the correct version's order. Construct the two commitments. Choose fresh rcv_0 , rcv_1 and α_0, α_1 , retrying forbidden identity randomised keys.
5. Form the net commitments

$$cv_0 = [30,000]V + [rcv_0]R, \quad cv_1 = [-20,000]V + [rcv_1]R.$$

Set the bundle public balance to 10,000.

6. Encrypt both notes and optional outgoing recovery records. Assemble the Action effecting fields, shared flags and anchor. Generate the aggregate proof from the ordered witnesses and instances.
7. Recompute the complete transaction signature digest. Sign both Action keys and the binding key

$$\text{bvk} = \text{cv}_0 + \text{cv}_1 - [10,000]V = [\text{rcv}_0 + \text{rcv}_1]R.$$

8. Serialize and run the complete local verification checks before submission. Retain the transaction identity and private output metadata needed for recovery.

The public Action count is two; it does not label the dummy spend. The public balance reveals the fee for this all-shielded transaction. Bob detects his note through incoming decryption, not from an explicit recipient field. Alice detects change under internal scope and later recognises consumption of her original note through its nullifier. Once mined, both commitments occupy definite positions in the selected branch's pool tree. A later spend proves membership without disclosing those positions.

10 Security boundaries

The construction supports a precise conditional argument, not a blanket claim that a transaction reveals nothing except validity.

10.1 What is mathematical

The following conclusions are identities or elementary implications: honest key agreement gives the same shared point; the randomised secret and public key match; correctly balanced value commitments cancel to the binding key; a nonzero bounded old value activates membership; bounded modular equality lifts to integer balance; and identical accepted nullifiers are rejected by the ledger's uniqueness rule.

These facts do not establish that a malicious prover cannot violate the premises. The step from an accepting proof to a witness uses knowledge soundness, and the step from a signature to authorised spending uses a signature security argument with the relevant related-key structure.

10.2 What is assumed or imported

The application uses the commitment binding and exceptional-case assumptions of the *Crypto Guide*, “Groups, commitments, and Sinsemilla”; the expansion and nullifier-PRF assumptions of “Hash functions and key derivation”; the signature contract of “RedPallas signatures”; and the proof contract of the *Halo 2 Guide*, “Knowledge soundness and zero knowledge”. For extracted malicious witnesses, a balance proof must compose those contracts with the binding-signature argument and the ledger's integer bounds.

Nullifier unlinkability additionally depends on the distribution of honestly generated note data and the appropriate PRF- or group-based argument. The protocol design gives alternative assumption routes; it does not make unlinkability unconditional when viewing keys, note data or addresses are disclosed.

Zero knowledge concerns the full joint proof transcript, with its public instances fixed. It does not hide the public instances, signatures, ciphertext lengths, pool balances or network behaviour. Marginally random-looking columns or commitments are not a substitute for the simulator and composition hypotheses of the proof system. This volume imports that proof-system contract rather than claiming a new complete simulator for deployed Halo 2.

10.3 What remains visible or unavailable

A chain observer sees transaction shape, pool membership, public balances, anchors, nullifiers, randomised keys, output commitments and ciphertexts. Public transfers, timings, repeated external addresses and application disclosures may correlate them. Coinbase plaintexts have the explicit public-recovery exception. A recipient learns its received note and sender-chosen memo, not an authenticated real-world sender identity supplied by the payment protocol.

A malicious sender can withhold delivery or outgoing recovery while creating a consensus-valid output. A malicious wallet server can omit data or present an incomplete branch view unless the client has an independent authentication and availability strategy. A reorganisation can remove a valid payment from the selected branch. None of these events contradicts proof zero knowledge or commitment binding.

Construction evidence. The key, note, nullifier, encryption, tree and circuit implementations are orchard 0.15.0 files `src/keys.rs`, `src/note.rs`, `src/note/nullifier.rs`, `src/note/encryption.rs`, `src/tree.rs` and `src/circuit.rs`. `Cargo.toml` identifies this registry release as version 0.15.0; its `.cargo_vcs_info.json` records source revision 8995ee7e. The development checkout bef8a27 is a separate source, not that release. The versioned digest rules are ZIP 244 and ZIP 229, implemented by `zcash_primitives/src/transaction/txid.rs` and `zcash_primitives/src/transaction/sighash_v5.rs` with the versioned extension paths. The companion check verifies the example's arithmetic, byte and bit widths, personalisation lengths and proof-size formula. It does not generate a production transaction or certify the imported cryptographic assumptions.